Sorting Algorithms and Practice Questions

1. Bubble Sort

Bubble Sort is one of the simplest sorting algorithms. It works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. The largest element "bubbles up" to its correct position in each pass.

How Bubble Sort Works:

- Compare adjacent elements in the array
- If the first element is greater than the second, swap them
- Continue this process for the entire array
- Repeat until no more swaps are needed

Time Complexity:

- Best Case: O(n) - when array is already sorted
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$ - when array is sorted in reverse order

Java Implementation:

java

```java
public class BubbleSort {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;

            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap elements
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }

            // If no swapping occurred, array is sorted
            if (!swapped) {
                break;
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        System.out.println("Original array:");
        printArray(arr);

        bubbleSort(arr);

        System.out.println("Sorted array:");
        printArray(arr);
    }

    public static void printArray(int[] arr) {
        for (int value : arr) {
            System.out.print(value + " ");
        }
        System.out.println();
```

```
    }
}
```

2. Selection Sort

Selection Sort works by finding the minimum element from the unsorted portion and placing it at the beginning. It divides the array into sorted and unsorted regions.

How Selection Sort Works:

- Find the minimum element in the unsorted array
- Swap it with the first element of the unsorted portion
- Move the boundary between sorted and unsorted portions
- Repeat until the entire array is sorted

Time Complexity:

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Java Implementation:

```java
public class SelectionSort {
    public static void selectionSort(int[] arr) {
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            // Find minimum element in remaining unsorted array
            int minIndex = i;

            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            // Swap the minimum element with the first element
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 25, 12, 22, 11};

        System.out.println("Original array:");
        printArray(arr);

        selectionSort(arr);

        System.out.println("Sorted array:");
        printArray(arr);
    }

    public static void printArray(int[] arr) {
        for (int value : arr) {
            System.out.print(value + " ");
        }
        System.out.println();
    }
}
```

3. Insertion Sort

Insertion Sort builds the sorted array one element at a time. It takes each element from the unsorted portion and inserts it into its correct position in the sorted portion.

How Insertion Sort Works:

- Start with the second element (assume first is sorted)
- Compare the current element with elements in the sorted portion
- Shift larger elements to the right
- Insert the current element in its correct position
- Repeat for all elements

Time Complexity:

- Best Case: $O(n)$ - when array is already sorted
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$ - when array is sorted in reverse order

Java Implementation:

```java
public class InsertionSort {
    public static void insertionSort(int[] arr) {
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            // Move elements greater than key one position ahead
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j = j - 1;
            }

            // Insert key at its correct position
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        int[] arr = {12, 11, 13, 5, 6};

        System.out.println("Original array:");
        printArray(arr);

        insertionSort(arr);

        System.out.println("Sorted array:");
        printArray(arr);
    }

    public static void printArray(int[] arr) {
        for (int value : arr) {
            System.out.print(value + " ");
        }
        System.out.println();
    }
}
```

Practice Questions on Sorting Algorithms

Question 1: Algorithm Identification

Given the array [5, 2, 8, 1, 9], trace through one complete pass of each sorting algorithm and identify which algorithm produces the following intermediate result after the first pass: [2, 5, 1, 8, 9]

Question 2: Efficiency Analysis

Compare the performance of Bubble Sort, Selection Sort, and Insertion Sort on:

- A nearly sorted array
- A reverse sorted array
- A randomly ordered array

Question 3: Implementation Modification

Modify the Bubble Sort algorithm to sort in descending order instead of ascending order.

Quiz Questions

1. What is the worst-case time complexity of Bubble Sort? a) O(n) b) O(n log n) c) $O(n^2)$ d) $O(n^3)$

Answer: c) $O(n^2)$

2. In Selection Sort, what happens in each pass? a) The largest element bubbles to the end b) Elements are inserted in their correct positions c) The minimum element is selected and placed at the beginning of unsorted portion d) Elements are compared with their adjacent elements

Answer: c) The minimum element is selected and placed at the beginning of unsorted portion

3. Which sorting algorithm performs best on nearly sorted data? a) Bubble Sort b) Selection Sort c) Insertion Sort d) All perform equally

Answer: c) Insertion Sort

4. How many comparisons does Selection Sort make for an array of size n? a) n b) n-1 c) n(n-1)/2 d) n^2

Answer: c) n(n-1)/2

5. In Insertion Sort, where does the algorithm start comparing elements? a) From the first element b) From the second element c) From the last element d) From the middle element

Answer: b) From the second element

Practice Coding Questions

1. Implement a modified Bubble Sort that counts the number of swaps made during the sorting process. Return both the sorted array and the swap count.

2. Write a program that implements Selection Sort for sorting an array of strings in alphabetical order.

3. Create an Insertion Sort implementation that can sort an array in both ascending and descending order based on a boolean parameter.

4. Implement a hybrid sorting method that uses Insertion Sort for small arrays (size ≤ 10) and Bubble Sort for larger arrays.

5. Write a program that compares the performance of all three sorting algorithms by measuring the time taken to sort the same array of 1000 random integers.